

Competitive Programming

Section 28 Editorial: DP I Mixed Contest

Competitive Programming Course

2026-08-09

Table of contents

1	Category coverage	5
2	Editorial overview	6
3	A. Training Schedule	7
3.1	Restatement	7
3.2	Constraints and traps	7
3.3	Observations	7
3.4	Algorithm	7
3.5	Correctness proof	8
3.6	Complexity	8
3.7	Full C++ solution	8
3.8	Full Python solution	8
3.9	Common wrong answers	9
3.10	Tests that matter	9
4	B. Two Carts	10
4.1	Restatement	10
4.2	Constraints and traps	10
4.3	Observations	10
4.4	Algorithm	10
4.5	Correctness proof	11
4.6	Complexity	11
4.7	Full C++ solution	11
4.8	Full Python solution	12
4.9	Common wrong answers	12
4.10	Tests that matter	13
5	C. Prerequisite Tree	14
5.1	Restatement	14
5.2	Constraints and traps	14
5.3	Observations	14
5.4	Algorithm	14
5.5	Correctness proof	15
5.6	Complexity	15
5.7	Full C++ solution	15
5.8	Full Python solution	16
5.9	Common wrong answers	17
5.10	Tests that matter	17
6	Testing discussion	18
7	What to carry forward	19

8	D. Circular Exhibition	20
8.1	Restatement and reduction	20
8.2	Constraints and traps	20
8.3	Worked trace	20
8.4	Algorithm	21
8.5	Correctness proof	21
8.6	Complexity	21
8.7	Full C++ solution	21
8.8	C++ implementation notes	22
8.9	Full Python solution	22
8.10	Python implementation notes	23
8.11	Traps and tests	23
9	Stress testing	24
10	What to carry forward	25
11	E. One Diagonal Path	26
11.1	Restatement and state design	26
11.2	Transitions	26
11.3	Correctness proof	26
11.4	Complexity and implementation notes	27
11.5	Tests that matter	27
11.6	Full C++ solution	27
11.7	Full Python solution	28
12	F. Matrix Chain	29
12.1	Why parenthesization matters	29
12.2	Recurrence and evaluation order	29
12.3	Correctness proof	29
12.4	Complexity and pitfalls	30
12.5	Tests that matter	30
12.6	Full C++ solution	30
12.7	Full Python solution	31
13	G. Hamiltonian Route	32
13.1	Constraint-driven recognition	32
13.2	Transition	32
13.3	Correctness proof	32
13.4	Complexity and implementation notes	33
13.5	Tests that matter	33
13.6	Full C++ solution	33
13.7	Full Python solution	34
14	H. Sliding Jump Cost	36
14.1	Derive the slow recurrence first	36
14.2	Maintain the transition minimum	36
14.3	Correctness proof	36
14.4	Complexity, pitfalls, and tests	37
14.5	Full C++ solution	37
14.6	Full Python solution	38

15 I. Grouped Cargo	39
15.1 The decision unit is a group	39
15.2 Correctness proof	39
15.3 Complexity, pitfalls, and tests	39
15.4 Full C++ solution	40
15.5 Full Python solution	40
16 J. Minimum Starting Energy	42
16.1 Why a forward maximum is the wrong summary	42
16.2 Evaluation order and compression	42
16.3 Correctness proof	42
16.4 Complexity, pitfalls, and tests	43
16.5 Full C++ solution	43
16.6 Full Python solution	44
17 K. Last Crystal Removed	45
17.1 Choosing the first removal does not isolate intervals	45
17.2 Correctness proof	45
17.3 Complexity, pitfalls, and tests	45
17.4 Full C++ solution	46
17.5 Full Python solution	46
18 L. Weighted Tree Guards	48
18.1 A Boolean state loses necessary boundary information	48
18.2 Enforcing at least one guarded child	48
18.3 Correctness proof	48
18.4 Complexity, pitfalls, and tests	49
18.5 Full C++ solution	49
18.6 Full Python solution	50
19 DP I closing checklist	53

1 Category coverage

Category	Coverage problem
one-dimensional DP	A. Training Schedule; H. Sliding Jump Cost
knapsack and subset DP	B. Two Carts; I. Grouped Cargo
grid DP	E. One Diagonal Path; J. Minimum Starting Energy
interval DP	F. Matrix Chain; K. Last Crystal Removed
tree DP	C. Prerequisite Tree; L. Weighted Tree Guards
bitmask DP	G. Hamiltonian Route

All twelve problems are self-contained local packages and are included by this section's checker.
D. Circular Exhibition remains additional one-dimensional boundary-case practice.

2 Editorial overview

Problem	Main idea	State
A. Training Schedule	previous action DP	<code>dp[last]</code>
B. Two Carts	2D 0/1 knapsack	<code>dp[a][b]</code>
C. Prerequisite Tree	tree knapsack	<code>dp[u][budget]</code>
D. Circular Exhibition	split a cyclic boundary case	<code>dp[index][taken]</code>
E. One Diagonal Path	grid DP with resource state	<code>dp[row][column][used]</code>
F. Matrix Chain	interval split DP	<code>dp[left][right]</code>
G. Hamiltonian Route	endpoint-and-mask DP	<code>dp[mask][last]</code>
H. Sliding Jump Cost	sliding transition minimum	<code>dp[index]</code> plus deque
I. Grouped Cargo	group-layer knapsack	<code>dp[weight]</code>
J. Minimum Starting Energy	reverse requirement DP	<code>need[row][column]</code>
K. Last Crystal Removed	choose the final interval action	<code>dp[left][right]</code>
L. Weighted Tree Guards	parent-dependence tree DP	<code>dp[node][state]</code>

3 A. Training Schedule

3.1 Restatement

Each day has zero, one, or two available activities. You can do at most one activity per day, and cannot repeat the same activity on consecutive days. Minimize rest days.

3.2 Constraints and traps

n is large, so trying all schedules is impossible. The only memory needed from the past is yesterday's activity.

The value 3 means both activities are available, not a third activity.

3.3 Observations

If yesterday's action is known, today's legal actions are determined:

```
rest is always legal
C++ is legal if available and yesterday was not C++
Python is legal if available and yesterday was not Python
```

3.4 Algorithm

```
dp[rest] = 0
dp[cpp] = INF
dp[python] = INF

for each day:
    next[rest] = min(dp) + 1
    if C++ available:
        next[cpp] = min(dp[rest], dp[python])
    if Python available:
        next[python] = min(dp[rest], dp[cpp])
    dp = next

print min(dp)
```

3.5 Correctness proof

Invariant. After processing a prefix of days, `dp[last]` is the minimum number of rest days among valid schedules for that prefix whose last action is `last`.

The empty prefix has cost 0 with previous action treated as rest. For a new day, every valid schedule either rests, does C++, or does Python. The transition checks exactly the legal previous actions for each choice and adds one rest day only in the rest case. Therefore it keeps the minimum cost for each possible last action. After all days, the best final last action gives the answer. $\square$

3.6 Complexity

```
Time: O(n)
Memory: O(1)
```

3.7 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    if (!(cin >> n)) return 0;
    const int INF = 1e9;
    array<int, 3> dp = {0, INF, INF};
    for (int day = 0; day < n; day++) {
        int available;
        cin >> available;
        array<int, 3> ndp = {min({dp[0], dp[1], dp[2]}) + 1, INF, INF};
        if (available & 1) ndp[1] = min(dp[0], dp[2]);
        if (available & 2) ndp[2] = min(dp[0], dp[1]);
        dp = ndp;
    }
    cout << min({dp[0], dp[1], dp[2]}) << '\n';
    return 0;
}
```

3.8 Full Python solution

```

import sys

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    n = data[0]
    days = data[1 : 1 + n]
    inf = 10**9
    dp = [0, inf, inf]
    for available in days:
        ndp = [min(dp) + 1, inf, inf]
        if available & 1:
            ndp[1] = min(dp[0], dp[2])
        if available & 2:
            ndp[2] = min(dp[0], dp[1])
        dp = ndp
    print(min(dp))

if __name__ == "__main__":
    main()

```

3.9 Common wrong answers

Treating value 3 as a separate activity.
 Allowing the same activity on consecutive days.
 Forgetting that resting resets the previous activity.

3.10 Tests that matter

All days unavailable.
 All days only one same activity.
 Alternating forced activities.
 Days where both activities are available.

4 B. Two Carts

4.1 Restatement

Each item can go in cart A, cart B, or be skipped. Each cart has its own weight capacity. Maximize total value.

4.2 Constraints and traps

This is not ordinary one-capacity knapsack. A state must remember both used capacities.

Each item can be used at most once. Updating capacities in increasing order can reuse the same item.

4.3 Observations

After processing some items, the future only depends on:

```
used weight in cart A
used weight in cart B
best value achieved with those weights
```

4.4 Algorithm

```
dp[a][b] = best value with a weight in A and b weight in B

for each item (w, v):
    for a from A down to 0:
        for b from B down to 0:
            put in A if possible
            put in B if possible

answer = max over all dp[a][b]
```

4.5 Correctness proof

Invariant. After processing the first i items, $dp[a][b]$ is the maximum value obtainable using exactly a weight in cart A and b weight in cart B.

For item $i+1$, any optimal solution either skips it, puts it in A, or puts it in B. Skipping is represented by keeping the old value. The two update cases represent placing the item into a cart if capacity allows. Decreasing iteration ensures each update uses a state from before the current item was placed, so the item cannot be counted twice. Thus the invariant is preserved. Taking the maximum over capacities gives the best value with capacity at most the limits. $\square$

4.6 Complexity

Time: $O(n A B)$

Memory: $O(A B)$

4.7 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, cap_a, cap_b;
    if (!(cin >> n >> cap_a >> cap_b)) return 0;
    vector<pair<int, long long>> items(n);
    for (auto &[w, v] : items) cin >> w >> v;

    vector<vector<long long>> dp(cap_a + 1, vector<long long>(cap_b + 1, 0));
    for (auto [weight, value] : items) {
        for (int a = cap_a; a >= 0; a--) {
            for (int b = cap_b; b >= 0; b--) {
                long long cur = dp[a][b];
                if (a + weight <= cap_a) {
                    dp[a + weight][b] = max(dp[a + weight][b], cur + value);
                }
                if (b + weight <= cap_b) {
                    dp[a][b + weight] = max(dp[a][b + weight], cur + value);
                }
            }
        }
    }

    long long answer = 0;
```

```

for (int a = 0; a <= cap_a; a++) {
    for (int b = 0; b <= cap_b; b++) answer = max(answer, dp[a][b]);
}
cout << answer << '\n';
return 0;
}

```

4.8 Full Python solution

```

import sys

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    it = iter(data)
    n = next(it)
    cap_a = next(it)
    cap_b = next(it)
    items = [(next(it), next(it)) for _ in range(n)]
    dp = [[0] * (cap_b + 1) for _ in range(cap_a + 1)]
    for weight, value in items:
        for a in range(cap_a, -1, -1):
            for b in range(cap_b, -1, -1):
                cur = dp[a][b]
                if a + weight <= cap_a:
                    dp[a + weight][b] = max(dp[a + weight][b], cur + value)
                if b + weight <= cap_b:
                    dp[a][b + weight] = max(dp[a][b + weight], cur + value)
    print(max(max(row) for row in dp))

if __name__ == "__main__":
    main()

```

4.9 Common wrong answers

Using one capacity equal to A+B; that loses which cart has space.
 Updating capacities upward and reusing one item.
 Forgetting the skip option.
 Returning only dp[A][B] when the best solution may leave unused capacity.

4.10 Tests that matter

```
An item too heavy for either cart.  
Two items each exactly fill a cart.  
Best answer leaves unused capacity.  
High values that require 64-bit sums.
```

5 C. Prerequisite Tree

5.1 Restatement

Lessons form a rooted tree. Selecting a lesson requires selecting its parent. Each lesson has time and value. Choose lessons with total time at most B and maximum total value.

5.2 Constraints and traps

The tree condition means a child cannot be selected independently. Ordinary knapsack over all lessons would allow invalid choices.

The root does not have to be selected; choosing no lesson is valid and has value 0.

5.3 Observations

If node u is selected, then each child subtree can either contribute nothing or contribute a valid selected set rooted at that child.

This is exactly a knapsack merge over children.

5.4 Algorithm

```
build children lists
build postorder from root

for u in postorder:
    dp[u][time[u]] = value[u]
    for child v:
        merged = dp[u]
        for used budget in u's current table:
            for add budget in child's table:
                merged[used + add] =
                    max(merged[used + add], dp[u][used] + dp[v][add])
        dp[u] = merged

answer = max(0, max_b dp[root][b])
```

5.5 Correctness proof

Invariant. After processing some children of u , $dp[u][b]$ is the maximum value of a valid prerequisite-respecting selected set inside u 's subtree, using exactly b time, requiring that u is selected, and using only the already processed child subtrees.

Initialization with only u selected is correct: time $time[u]$, value $value[u]$.

When merging a child v , a valid solution either takes nothing from v , which is kept by copying $dp[u]$, or takes a valid set from v 's subtree. Because $dp[v][add]$ requires that v is selected, and u is already selected, the prerequisite edge from v to u is satisfied. Trying all budget splits and taking the maximum gives the best merged table.

By postorder, every child table is correct before it is merged. After all children are processed, the invariant gives the correct table for u . Applying this to the root and taking the best budget at most B , plus the option of choosing nothing, gives the answer. $\square$

5.6 Complexity

Time: $O(n B^2)$

Memory: $O(n B)$

5.7 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, budget;
    if (!(cin >> n >> budget)) return 0;
    vector<int> need(n);
    vector<long long> value(n);
    for (int i = 0; i < n; i++) cin >> need[i] >> value[i];
    vector<vector<int>> children(n);
    for (int child = 1; child < n; child++) {
        int parent;
        cin >> parent;
        children[parent - 1].push_back(child);
    }

    vector<int> order = {0};
    for (int i = 0; i < (int)order.size(); i++) {
        int u = order[i];
        for (int v : children[u]) order.push_back(v);
    }
}
```

```

}

const long long NEG = -(1LL << 60);
vector<vector<long long>> dp(n, vector<long long>(budget + 1, NEG));
for (int idx = n - 1; idx >= 0; idx--) {
    int u = order[idx];
    if (need[u] <= budget) dp[u][need[u]] = value[u];
    for (int v : children[u]) {
        vector<long long> merged = dp[u];
        for (int used = 0; used <= budget; used++) {
            if (dp[u][used] == NEG) continue;
            for (int add = 1; used + add <= budget; add++) {
                if (dp[v][add] == NEG) continue;
                merged[used + add] = max(merged[used + add], dp[u][used] + dp[v][add]);
            }
        }
        dp[u].swap(merged);
    }
}

long long answer = 0;
for (long long x : dp[0]) answer = max(answer, x);
cout << answer << '\n';
return 0;
}

```

5.8 Full Python solution

```

import sys

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    it = iter(data)
    n = next(it)
    budget = next(it)
    time = [0] * n
    value = [0] * n
    for i in range(n):
        time[i] = next(it)
        value[i] = next(it)
    children = [[] for _ in range(n)]
    for child in range(1, n):
        parent = next(it) - 1
        children[parent].append(child)

```

```

order = [0]
for u in order:
    order.extend(children[u])

neg = -10**30
dp = [[neg] * (budget + 1) for _ in range(n)]
for u in reversed(order):
    if time[u] <= budget:
        dp[u][time[u]] = value[u]
    for v in children[u]:
        merged = dp[u][:]
        for used in range(budget + 1):
            if dp[u][used] <= neg // 2:
                continue
            for add in range(1, budget - used + 1):
                if dp[v][add] <= neg // 2:
                    continue
                candidate = dp[u][used] + dp[v][add]
                if candidate > merged[used + add]:
                    merged[used + add] = candidate
        dp[u] = merged
answer = max(0, max(dp[0]))
print(answer)

if __name__ == "__main__":
    main()

```

5.9 Common wrong answers

Choosing a child without its parent.
 Merging child tables before they are computed.
 Forgetting that taking nothing from a child is allowed.
 Returning only exact budget B instead of at most B.
 Failing when the root is too expensive and the answer should be 0.

5.10 Tests that matter

Single root too expensive.
 A chain where selecting the deepest node requires all ancestors.
 A star where child choices compete for budget.
 Best answer leaves unused budget.

6 Testing discussion

For this mixed contest, the random generators use brute force where practical:

```
schedule: direct DP cross-check  
two carts: try all  $3^n$  choices for small  $n$   
tree prerequisites: enumerate all subsets for small  $n$ 
```

That is the right habit for upsolving DP. First prove the transition, then stress it against brute force on tiny random cases.

7 What to carry forward

The main skill from this section is diagnostic:

```
If only the previous action matters, keep only the previous action.  
If two resources constrain the future, keep both.  
If choices live under a tree, solve children before parents.  
If a budget is attached to each subtree, merge budgets like knapsack.
```

8 D. Circular Exhibition

8.1 Restatement and reduction

Choose non-adjacent booths on a circle for maximum reward. On a line, let `best[i]` be the best reward through position `i`; then:

```
best[i] = max(best[i-1], best[i-2] + a[i])
```

The circle only adds one conflict: the first and last booths cannot both be chosen. Every valid choice either excludes the last booth or excludes the first booth. Solve those two lines and take the larger answer. For one booth, return `max(0, a[0])` directly.

For sample `2 7 9 3 1`, excluding the last permits `2 + 9 = 11`; no valid circular selection is better.

8.2 Constraints and traps

`n` can be 200000, so subset enumeration and a two-dimensional DP are out of scope. Rewards may be negative and can be as large as 10^9 in magnitude.

```
Selecting no booth is allowed, so the answer is never negative.  
Booth 1 conflicts with booth n as well as with booth 2.  
n = 1 is not two ordinary line cases; handle it directly.  
Use 64-bit totals in C++.
```

8.3 Worked trace

For the sample `[2, 7, 9, 3, 1]`, first exclude the last booth and solve `[2, 7, 9, 3]`:

```
value:      2  7  9  3  
best so far: 2  7 11 11
```

Now exclude the first booth and solve `[7, 9, 3, 1]`:

```
value:      7  9  3  1  
best so far: 7  9 10 10
```

The larger case result is 11. For `n = 1`, say `[-4]`, neither line case is used: the legal choices are take `-4` or take nothing, giving 0.

8.4 Algorithm

```
line(l, r):
    previous_two = previous_one = 0
    for i from l to r:
        current = max(previous_one, previous_two + a[i])
        shift values forward
    return previous_one

if n == 1: answer = max(0, a[0])
else:     answer = max(line(0, n-2), line(1, n-1))
```

8.5 Correctness proof

Lemma 1. `line(l,r)` returns the maximum reward from non-adjacent booths in that line interval.

Proof. For each position, an optimal choice either skips it, giving the previous optimum, or takes it and must combine it with the optimum ending two positions earlier. The recurrence considers both cases. Induction over the line positions proves the result. $\square$

Lemma 2. Every valid circular selection belongs to at least one of the two line cases.

Proof. A valid selection cannot contain both first and last booths. Thus it omits the last (first line case), or omits the first (second line case), or both. $\square$

Theorem. The algorithm prints the maximum valid circular reward.

Proof. Lemma 2 places every valid selection in a solved line case. Lemma 1 optimizes each case, so their maximum is optimal. The single-booth branch has exactly the allowed choices: take it or take nothing. $\square$

8.6 Complexity

```
Time:  $O(n)$ 
Memory:  $O(1)$ 
```

8.7 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

long long line(const vector<long long>& a, int left, int right) {
    long long two_back = 0, one_back = 0;
    for (int i = left; i <= right; ++i) {
        long long current = max(one_back, two_back + a[i]);
```

```

        two_back = one_back;
        one_back = current;
    }
    return one_back;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n; cin >> n;
    vector<long long> a(n);
    for (long long& x : a) cin >> x;
    if (n == 1) cout << max(0LL, a[0]) << "\\n";
    else cout << max(line(a, 0, n - 2), line(a, 1, n - 1)) << "\\n";
}

```

8.8 C++ implementation notes

Pass the original vector plus inclusive `left`, `right` indices to `line`; this does not copy either case. Initialize both rolling values to zero so skipping all negative booths stays legal.

8.9 Full Python solution

```

import sys

def line(values: list[int], left: int, right: int) -> int:
    two_back = one_back = 0
    for index in range(left, right + 1):
        two_back, one_back = one_back, max(one_back, two_back + values[index])
    return one_back

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    values = data[1:]
    if len(values) == 1:
        print(max(0, values[0]))
    else:
        print(max(line(values, 0, len(values) - 2),
                  line(values, 1, len(values) - 1)))

if __name__ == "__main__":
    main()

```

8.10 Python implementation notes

Avoid `values[:-1]` and `values[1:]`: each slice creates a new list. Passing the same list with endpoint indices preserves the advertised $O(1)$ auxiliary DP memory (besides the unavoidable input list).

8.11 Traps and tests

- Treating the circle as a line allows selecting both endpoints.
- Forgetting the one-booth special case makes its two line intervals empty.
- Negative rewards mean taking no booth can be best.
- Test one booth, all equal rewards, endpoint-heavy values, and all negatives.

9 Stress testing

For at most 18 booths, enumerate every subset and reject any subset containing adjacent circular positions. This independent oracle checks the two-case DP. The local generator also includes 200000 booths of reward one, whose direct answer is 100000.

10 What to carry forward

When a cycle blocks a simple path DP only at its endpoints, split on one endpoint's inclusion/exclusion and solve the remaining path cases.

11 E. One Diagonal Path

11.1 Restatement and state design

We move from the top-left cell to the bottom-right using right and down moves, but exactly one move must be diagonal down-right. Cell values may be negative, so an unreachable state cannot be represented by zero.

Position alone does not describe a partial route: two routes reaching the same cell differ in whether the mandatory diagonal is still available. Add one bit:

```
dp[r][c][0] = best sum reaching (r,c) without a diagonal
dp[r][c][1] = best sum reaching (r,c) after exactly one diagonal
```

The answer is state 1 at the destination; taking the maximum over both states would incorrectly permit no diagonal.

11.2 Transitions

For either `used` value, a route may arrive from above or from the left:

```
dp[r][c][used] =
    value[r][c] + max(dp[r-1][c][used], dp[r][c-1][used])
```

The diagonal transition is different:

```
dp[r][c][1] =
    max(dp[r][c][1], dp[r-1][c-1][0] + value[r][c])
```

It must originate from state zero; a transition from state one would use a second diagonal. Initialize only `dp[0][0][0]` with the starting cell.

11.3 Correctness proof

We prove by row-major induction that each state equals the maximum sum of all routes satisfying its definition. The base state represents the unique zero-move route at $(0,0)$.

Any route reaching (r,c) ends with exactly one of three moves. If the final move is down or right, removing it leaves a route to the corresponding predecessor with the same `used` bit; the first transition considers it. If the final move is diagonal, the preceding route used no diagonal and the second transition considers it. These cases are exhaustive and disjoint. Conversely, appending the indicated move to any reachable predecessor produces a valid route for the destination state. Taking the maximum therefore gives the optimal sum for each state. In particular, destination state one is the best route using exactly one diagonal.

11.4 Complexity and implementation notes

There are $2nm$ states and constant work per state, so time is $O(nm)$. Keeping only the previous and current row reduces memory from $O(nm)$ to $O(m)$.

Use a genuine negative-infinity sentinel and never add a cell value to an unreachable state. With negative grid values, zero is not a safe sentinel. If $n = 1$ or $m = 1$, no diagonal move is possible and the output is -1 .

11.5 Tests that matter

```
1 x 1 and 1 x m grids -> impossible.
2 x 2 -> the diagonal route is forced.
All-negative values -> catches zero initialization.
A tempting second diagonal -> verifies the used bit.
Best route delays the diagonal until near the destination.
```

For small dimensions, enumerate every legal move sequence and retain only those containing exactly one diagonal.

11.6 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    vector<vector<ll>> a(n, vector<ll>(m));
    for (auto& row : a) for (ll& x : row) cin >> x;
    const ll NEG = -(1LL << 62);
    vector<array<ll, 2>> previous(m, {NEG, NEG});
    for (int r = 0; r < n; ++r) {
        vector<array<ll, 2>> current(m, {NEG, NEG});
        for (int c = 0; c < m; ++c) {
            if (r == 0 && c == 0) current[c][0] = a[r][c];
            for (int used = 0; used < 2; ++used) {
                ll best = current[c][used];
                if (r && previous[c][used] != NEG) {
                    best = max(best, previous[c][used] + a[r][c]);
                }
                if (c && current[c - 1][used] != NEG) {
                    best = max(best, current[c - 1][used] + a[r][c]);
                }
            }
        }
    }
}
```

```

        current[c][used] = best;
    }
    if (r && c && previous[c - 1][0] != NEG) {
        current[c][1] = max(current[c][1], previous[c - 1][0] + a[r][c]);
    }
}
previous.swap(current);
}
cout << (previous[m - 1][1] == NEG ? -1 : previous[m - 1][1]) << '\n';
}

```

11.7 Full Python solution

```

import sys
def main():
    data = list(map(int, sys.stdin.buffer.read().split()))
    n, m = data[:2]
    values = data[2:]
    grid = [values[r*m:(r+1)*m] for r in range(n)]
    neg = -10**30
    previous = [[neg, neg] for _ in range(m)]
    for r in range(n):
        current = [[neg, neg] for _ in range(m)]
        for c in range(m):
            if r == 0 and c == 0:
                current[c][0] = grid[r][c]
            for used in range(2):
                if r and previous[c][used] != neg:
                    current[c][used] = max(current[c][used], previous[c][used] + grid[r][c])
                if c and current[c-1][used] != neg:
                    current[c][used] = max(current[c][used], current[c-1][used] + grid[r][c])
            if r and c and previous[c-1][0] != neg:
                current[c][1] = max(current[c][1], previous[c-1][0] + grid[r][c])
        previous = current
    print(-1 if previous[-1][1] == neg else previous[-1][1])
if __name__ == "__main__":
    main()

```

12 F. Matrix Chain

12.1 Why parenthesization matters

Matrix multiplication is associative, but its cost depends on the order. If matrices A_i have dimensions $d[i-1] \times d[i]$, every product of matrices $1..r$ has fixed final dimensions $d[1-1] \times d[r]$, regardless of how that interval is parenthesized. This fixed boundary information makes an interval DP possible.

Define:

```
dp[l][r] = minimum scalar multiplications needed for A_l ... A_r
```

A single matrix needs no multiplication, so $dp[i][i] = 0$.

12.2 Recurrence and evaluation order

Every parenthesization of $[l, r]$ has a unique final multiplication, splitting after some k :

```
(A_l ... A_k) * (A_{k+1} ... A_r)
```

The two results have dimensions $d[l-1] \times d[k]$ and $d[k] \times d[r]$. Therefore:

```
dp[l][r] = min over l <= k < r of  
    dp[l][k] + dp[k+1][r] + d[l-1] * d[k] * d[r]
```

Fill intervals by increasing length so both smaller intervals are already known when evaluating a split.

12.3 Correctness proof

Take an optimal parenthesization of $[l, r]$ and let k be its final split. Its left and right subexpressions must themselves be optimally parenthesized; otherwise replacing either with a cheaper parenthesization improves the whole solution. Thus its cost appears among the recurrence candidates.

Conversely, for every candidate k , combine optimal parenthesizations for the two subintervals and perform their final multiplication. This constructs a valid parenthesization with exactly the candidate cost. The minimum over all final splits is therefore exactly optimal.

12.4 Complexity and pitfalls

There are $O(n^2)$ intervals and up to $O(n)$ splits per interval, giving $O(n^3)$ time and $O(n^2)$ memory. Products and totals require signed 64-bit integers under the stated dimensions.

Frequent mistakes include shifting the dimension indices incorrectly, using $d[k+1]$ for the shared dimension in one-based notation, initializing nontrivial intervals to zero, or filling by decreasing rather than increasing interval size.

12.5 Tests that matter

<code>n = 1</code>	<code>-> 0</code>
<code>Two matrices</code>	<code>-> one forced multiplication</code>
<code>10 30 5 60</code>	<code>-> split choice changes the answer</code>
<code>All dimensions equal</code>	<code>-> every parenthesization ties</code>
<code>Large dimensions</code>	<code>-> catches 32-bit overflow</code>

A recursive exhaustive enumeration of all split positions is a suitable oracle for small n .

12.6 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n; cin >> n;
    vector<ll> d(n + 1);
    for (ll& x : d) cin >> x;
    vector<vector<ll>> dp(n, vector<ll>(n));
    for (int length = 2; length <= n; ++length) {
        for (int l = 0; l + length <= n; ++l) {
            int r = l + length - 1;
            dp[l][r] = (1LL << 62);
            for (int k = l; k < r; ++k) {
                dp[l][r] = min(dp[l][r],
                               dp[l][k] + dp[k + 1][r] + d[l] * d[k + 1] * d[r + 1]);
            }
        }
    }
    cout << dp[0][n - 1] << '\n';
}
```

12.7 Full Python solution

```
import sys
def main():
    data=list(map(int,sys.stdin.buffer.read().split()))
    n=data[0];d=data[1:]
    dp=[[0]*n for _ in range(n)]
    for length in range(2,n+1):
        for l in range(n-length+1):
            r=l+length-1
            dp[l][r]=min(dp[l][k]+dp[k+1][r]+d[l]*d[k+1]*d[r+1] for k in range(l,r))
    print(dp[0][n-1])
if __name__=="__main__":main()
```

13 G. Hamiltonian Route

13.1 Constraint-driven recognition

Visiting every vertex exactly once suggests permutations, but $n!$ is already too large for $n = 18$. The important constraint is small n , which permits 2^n subsets. A subset records which vertices have been used; the final vertex records enough order information to extend the route.

Define:

```
dp[mask][u] = minimum cost of a route that
               starts at vertex 1,
               visits exactly the vertices in mask,
               and currently ends at u
```

The base state is $dp[1 \ll 0][0] = 0$.

13.2 Transition

From a reachable $(mask, u)$, try every vertex v not in $mask$. If the directed edge $u \rightarrow v$ exists, append it:

```
new_mask = mask | (1 << v)
dp[new_mask][v] =
    min(dp[new_mask][v], dp[mask][u] + cost[u][v])
```

The answer is $dp[(1 \ll n) - 1][n - 1]$. Although one may skip transitions into vertex n before the last step as an optimization, it is not required: because vertices cannot be revisited, a state that visits n early can never finish at n with the full mask.

13.3 Correctness proof

We prove by the number of set bits in $mask$ that every DP state stores the cheapest route matching its definition. The base state is the unique route containing only vertex 1.

For a larger state $(mask, v)$, every matching route has a unique penultimate vertex u . Removing its last edge gives a route represented by $(mask \text{ without } v, u)$, and the transition from that state considers the route's exact cost. Conversely, every transition appends an existing directed edge to an unvisited vertex, so it creates a route with no repeated vertex and the claimed mask and endpoint. Minimizing over all possible penultimate vertices therefore yields the optimal state value. The full-mask state ending at n is exactly the required Hamiltonian route.

13.4 Complexity and implementation notes

There are $n \cdot 2^n$ states and up to n outgoing choices per state, for $O(n^2 \cdot 2^n)$ time and $O(n \cdot 2^n)$ memory. Use 64-bit costs and an infinity value safely above every valid route cost.

Do not treat -1 as an edge weight; it means no edge. The matrix is directed, so `cost[u][v]` and `cost[v][u]` are unrelated. Ensure the start bit is present in every reachable mask.

13.5 Tests that matter

```
n = 2 with and without edge 1 -> 2.
A unique Hamiltonian route.
All vertices reachable, but no Hamiltonian route ending at n.
An asymmetric matrix where reversing an edge is invalid.
A cheap route that reaches n too early and becomes unusable.
Large edge costs -> verifies 64-bit accumulation.
```

For $n \leq 9$, enumerate all permutations of the intermediate vertices as an independent oracle.

13.6 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

using ll = long long;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    cin >> n;

    vector<vector<ll>> weight(n, vector<ll>(n));
    for (auto& row : weight)
        for (ll& value : row)
            cin >> value;

    const ll INF = 1LL << 62;
    const int mask_count = 1 << n;
    vector<vector<ll>> dp(mask_count, vector<ll>(n, INF));
    dp[1][0] = 0;

    for (int mask = 1; mask < mask_count; ++mask) {
        for (int u = 0; u < n; ++u) {
            if (dp[mask][u] == INF)
```

```

        continue;

    for (int v = 0; v < n; ++v) {
        if ((mask >> v & 1) || weight[u][v] < 0)
            continue;

        int next_mask = mask | (1 << v);
        dp[next_mask][v] = min(
            dp[next_mask][v],
            dp[mask][u] + weight[u][v]
        );
    }
}

ll answer = dp[mask_count - 1][n - 1];
cout << (answer == INF ? -1 : answer) << '\n';
}

```

13.7 Full Python solution

```

import sys

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    n = data[0]
    values = data[1:]
    weight = [
        values[row * n:(row + 1) * n]
        for row in range(n)
    ]

    infinity = 10**30
    mask_count = 1 << n
    dp = [[infinity] * n for _ in range(mask_count)]
    dp[1][0] = 0

    for mask in range(mask_count):
        for u in range(n):
            if dp[mask][u] == infinity:
                continue

            for v in range(n):
                if mask >> v & 1 or weight[u][v] < 0:
                    continue

                next_mask = mask | (1 << v)
                dp[next_mask][v] = min(

```

```

        dp[next_mask][v],
        dp[mask][u] + weight[u][v],
    )

    answer = dp[mask_count - 1][n - 1]
    print(-1 if answer == infinity else answer)

if __name__ == "__main__":
    main()

```

14 H. Sliding Jump Cost

14.1 Derive the slow recurrence first

Let $dp[i]$ be the minimum cost of a route ending at checkpoint i , including the cost of i . The final jump comes from one of the preceding k checkpoints, so

```
dp[0] = cost[0]
dp[i] = cost[i] + min(dp[j]) for max(0, i-k) <= j < i
```

This is a correct $O(nk)$ DP. The state is already small; the repeated scan of an overlapping window is the bottleneck.

14.2 Maintain the transition minimum

Keep candidate indices in a deque whose dp values are increasing.

Before computing $dp[i]$:

1. remove indices smaller than $i-k$ from the front;
2. the front now has the minimum DP value in the legal predecessor window;
3. after computing $dp[i]$, remove from the back every candidate with value at least $dp[i]$, then append i .

An older candidate with a no-smaller DP value is dominated by i : every future window containing the older index also contains i for at least as long. Negative checkpoint costs do not change this argument; the deque orders DP values, not raw costs.

14.3 Correctness proof

Assume all states before i equal the recurrence. Expired indices cannot jump to i and are removed. Every candidate removed from the back has a later candidate with no greater DP value, so it can never be the unique best choice for a future state. Thus the deque front is exactly a minimum over all legal predecessors of i , and the computed value equals the recurrence. Induction proves all DP values, including the destination, are optimal.

14.4 Complexity, pitfalls, and tests

Each index is inserted once and removed at most once from each end, giving $O(n)$ time and $O(n)$ memory. The DP array can also be reduced, but retaining it makes the invariant easier to inspect. Include the start and destination costs, expire before reading the front, and use 64-bit totals.

```
n = 1.  
k = 1 and k >= n.  
Negative costs that make a longer route cheaper.  
Equal DP values in the deque.
```

The oracle evaluates the direct $O(nk)$ recurrence.

14.5 Full C++ solution

```
#include <bits/stdc++.h>  
using namespace std;  
  
int main() {  
    ios::sync_with_stdio(false);  
    cin.tie(nullptr);  
  
    int n, k;  
    cin >> n >> k;  
    vector<long long> cost(n), dp(n);  
    for (long long& value : cost)  
        cin >> value;  
  
    deque<int> candidates;  
    dp[0] = cost[0];  
    candidates.push_back(0);  
  
    for (int i = 1; i < n; ++i) {  
        while (candidates.front() < i - k)  
            candidates.pop_front();  
  
        dp[i] = cost[i] + dp[candidates.front()];  
  
        while (!candidates.empty() && dp[candidates.back()] >= dp[i])  
            candidates.pop_back();  
        candidates.push_back(i);  
    }  
  
    cout << dp[n - 1] << '\n';  
}
```

14.6 Full Python solution

```
import collections
import sys

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    n, k = data[:2]
    cost = data[2:]
    dp = [0] * n
    candidates = collections.deque([0])
    dp[0] = cost[0]

    for i in range(1, n):
        while candidates[0] < i - k:
            candidates.popleft()

        dp[i] = cost[i] + dp[candidates[0]]

        while candidates and dp[candidates[-1]] >= dp[i]:
            candidates.pop()
        candidates.append(i)

    print(dp[-1])

if __name__ == "__main__":
    main()
```

15 I. Grouped Cargo

15.1 The decision unit is a group

Ordinary 0/1 knapsack processes one item at a time. Here that would lose the fact that two items from the same group are mutually exclusive.

After processing complete groups, define

```
dp[c] = maximum value obtainable with total weight at most c
```

For the next group, begin with `next = dp`, representing the choice of no item. For each item (`weight, value`) in the group and each old capacity `c`, update

```
next[c + weight] = max(next[c + weight], dp[c] + value)
```

Crucially, every transition reads from `dp`, the previous group layer. Reading from `next` could combine two items from the current group.

15.2 Correctness proof

Induct on the number of processed groups. A valid selection after the new group either chooses no item from it, covered by copying `dp`, or chooses exactly one specific item. Removing that item leaves a valid selection from all previous groups and is considered by its transition. Conversely, copied states choose nothing new, and every transition adds only one item to a previous-layer state, so no group contributes twice. Taking maxima therefore produces precisely the best valid value for every capacity.

15.3 Complexity, pitfalls, and tests

If T is the total number of items, time is $O(TW)$ and memory is $O(W)$. Copy once per group, not once per item. A backward capacity loop alone does not repair an in-place update: different items in the same group could still be combined.

```
One group with two individually attractive items.  
Skipping an entire group is optimal.  
Two groups contain items with equal weights.  
Capacity smaller than every item.
```

The oracle enumerates one choice—possibly none—from each group.

15.4 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int groups, capacity;
    cin >> groups >> capacity;
    vector<long long> dp(capacity + 1, 0);

    for (int group = 0; group < groups; ++group) {
        int count;
        cin >> count;
        vector<pair<int, long long>> items(count);
        for (auto& [weight, value] : items)
            cin >> weight >> value;

        vector<long long> next = dp;
        for (auto [weight, value] : items)
            for (int used = 0; used + weight <= capacity; ++used)
                next[used + weight] = max(next[used + weight],
                                           dp[used] + value);

        dp.swap(next);
    }

    cout << *max_element(dp.begin(), dp.end()) << '\n';
}
```

15.5 Full Python solution

```
import sys

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    index = 0
    groups, capacity = data[index:index + 2]
    index += 2
    dp = [0] * (capacity + 1)

    for _ in range(groups):
        count = data[index]
        index += 1
        items = []
```

```

    for _ in range(count):
        weight, value = data[index:index + 2]
        index += 2
        items.append((weight, value))

    next_dp = dp.copy()
    for weight, value in items:
        for used in range(capacity - weight + 1):
            next_dp[used + weight] = max(
                next_dp[used + weight], dp[used] + value)
    dp = next_dp

    print(max(dp))

if __name__ == "__main__":
    main()

```

16 J. Minimum Starting Energy

16.1 Why a forward maximum is the wrong summary

Two paths reaching the same cell can have different current energies and different lowest energies seen earlier. Keeping only the largest accumulated sum does not directly minimize the initial energy needed to survive every prefix.

Turn the question around:

```
need[r][c] = minimum energy required immediately before entering (r,c)
              to reach the destination without becoming negative
```

If the chosen next cell requires **after** energy, entering the current cell changes energy by `grid[r][c]`. Therefore

```
need[r][c] = max(0, after - grid[r][c])
```

Choose the smaller requirement of the right and down successors. At the destination, **after** is conceptually 0 after processing its value, giving `max(0, -grid[last])`.

16.2 Evaluation order and compression

Dependencies point right and down, so iterate rows and columns backward. A one-dimensional array stores the current row: before updating `need[c]` it is the value from below, while `need[c+1]` has already been updated for the cell to the right.

16.3 Correctness proof

At the destination the formula is exactly the least energy that remains nonnegative after applying its cell value. For another cell, every valid path chooses right or down next. By induction, the successor DP value is its exact required energy. Subtracting the current cell contribution and clamping at zero gives the exact requirement for that successor; taking the minimum considers both possible final choices. Thus every state and `need[0][0]` are optimal.

16.4 Complexity, pitfalls, and tests

Time is $O(nm)$ and compressed memory is $O(m)$. Clamp at zero because zero energy is valid but negative energy is not. Include the value of both endpoint cells, and use a wide type for long negative paths.

A single positive, zero, or negative cell.
All positive cells $\rightarrow$ answer 0.
A large reward after an unavoidable early loss.
Two routes trading an early loss against a later reward.

The oracle enumerates small paths and calculates the smallest initial energy required by each sequence of prefix sums.

16.5 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, m;
    cin >> n >> m;
    vector<vector<long long>> grid(n, vector<long long>(m));
    for (auto& row : grid)
        for (long long& value : row)
            cin >> value;

    const long long INF = 1LL << 62;
    vector<long long> need(m, INF);

    for (int row = n - 1; row >= 0; --row) {
        for (int column = m - 1; column >= 0; --column) {
            if (row == n - 1 && column == m - 1) {
                need[column] = max(0LL, -grid[row][column]);
                continue;
            }

            long long after = INF;
            if (row + 1 < n)
                after = min(after, need[column]);
            if (column + 1 < m)
                after = min(after, need[column + 1]);
            need[column] = max(0LL, after - grid[row][column]);
        }
    }
}
```

```
    cout << need[0] << '\n';  
}
```

16.6 Full Python solution

```
import sys  
  
def main() -> None:  
    data = list(map(int, sys.stdin.buffer.read().split()))  
    n, m = data[:2]  
    values = data[2:]  
    grid = [values[row * m:(row + 1) * m] for row in range(n)]  
    infinity = 10**30  
    need = [infinity] * m  
  
    for row in range(n - 1, -1, -1):  
        for column in range(m - 1, -1, -1):  
            if row == n - 1 and column == m - 1:  
                need[column] = max(0, -grid[row][column])  
                continue  
            after = infinity  
            if row + 1 < n:  
                after = min(after, need[column])  
            if column + 1 < m:  
                after = min(after, need[column + 1])  
            need[column] = max(0, after - grid[row][column])  
  
    print(need[0])  
  
if __name__ == "__main__":  
    main()
```

17 K. Last Crystal Removed

17.1 Choosing the first removal does not isolate intervals

If a crystal is removed first, its nearest remaining neighbors later depend on how both surrounding regions are removed. The natural-looking first action does not create independent subproblems.

Instead, fix two crystals `left` and `right` that remain present and ask which interior crystal is removed *last*. At that moment all other interior crystals are gone, so its neighbors are known exactly: `left` and `right`.

After adding permanent boundary values at indices 0 and $n+1$, define

```
dp[left][right] = maximum gain from removing every crystal
                  strictly between left and right
```

If `last` is the final interior crystal removed:

```
dp[left][right] = max over last of
    dp[left][last] + dp[last][right]
    + value[left] * value[last] * value[right]
```

Evaluate by increasing interval length.

17.2 Correctness proof

Every removal order for an interval has a unique final removed crystal. Before that final action, crystals on its left and right have been removed independently, giving the two smaller interval states; its final gain uses the fixed boundaries. Thus the recurrence considers every removal order with its exact gain. Conversely, combining optimal orders for both subintervals and then removing `last` is a legal order for the larger interval. Maximizing over `last` is therefore optimal.

17.3 Complexity, pitfalls, and tests

There are $O(n^2)$ intervals and $O(n)$ final choices per interval, for $O(n^3)$ time and $O(n^2)$ memory. The permanent boundary crystals are never removed. Use 64-bit multiplication before adding interval totals.

One crystal.
All values equal to one.
A small value between two large values.
Orders whose best first action is not locally largest.

The oracle tries every next removal on small tuples and memoizes the remaining sequence.

17.4 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    cin >> n;
    vector<long long> value(n + 2, 1);
    for (int i = 1; i <= n; ++i)
        cin >> value[i];

    // dp[left][right]: best gain after removing every crystal strictly
    // between the two still-present boundary indices left and right.
    vector<vector<long long>> dp(n + 2, vector<long long>(n + 2, 0));

    for (int gap = 2; gap <= n + 1; ++gap) {
        for (int left = 0; left + gap <= n + 1; ++left) {
            int right = left + gap;
            for (int last = left + 1; last < right; ++last) {
                dp[left][right] = max(
                    dp[left][right],
                    dp[left][last] + dp[last][right]
                    + value[left] * value[last] * value[right]);
            }
        }
    }

    cout << dp[0][n + 1] << '\n';
}
```

17.5 Full Python solution

```

import sys

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    n = data[0]
    value = [1] + data[1:] + [1]
    dp = [[0] * (n + 2) for _ in range(n + 2)]

    for gap in range(2, n + 2):
        for left in range(n + 2 - gap):
            right = left + gap
            dp[left][right] = max(
                dp[left][last] + dp[last][right]
                + value[left] * value[last] * value[right]
                for last in range(left + 1, right)
            )

    print(dp[0][n + 1])

if __name__ == "__main__":
    main()

```

18 L. Weighted Tree Guards

18.1 A Boolean state loses necessary boundary information

Knowing only whether u has a guard is insufficient. When u has no guard, it may already be protected by a child, or it may still require its parent to have a guard. Those cases impose different conditions above the subtree.

Root the tree and define three states:

```
dp[u][0] = u has a guard
dp[u][1] = u has no guard and is protected by a child
dp[u][2] = u is not yet protected and requires a guard at its parent
```

If u has a guard, each child may use any state because u protects a child waiting in state 2. If u waits for its parent, no child may have a guard (that would already protect u), and every child must be safely protected in state 1. If u is protected by a child, every child must be in state 0 or 1, and at least one must be in state 0.

18.2 Enforcing at least one guarded child

For state 1, first choose $\min(dp[v][0], dp[v][1])$ independently for every child. Then force one child into state 0 by adding the smallest penalty

```
dp[v][0] = min(dp[v][0], dp[v][1])
```

A leaf has no candidate child, so state 1 is impossible for it. At the root, state 2 is impossible because no parent exists.

18.3 Correctness proof

Proceed in postorder. The three states partition all possible relationships between a subtree root and its parent. Under a fixed state of u , child subtrees share no vertices or edges, so their optimal compatible costs add. The transitions list exactly the child states compatible with whether u has a guard and whether some child must guard it. Therefore, by induction, each state stores its minimum valid cost. The root must be guarded or protected by a child, so the answer is $\min(dp[root][0], dp[root][1])$.

18.4 Complexity, pitfalls, and tests

Every edge is inspected a constant number of times, giving $O(n)$ time and $O(n)$ memory. Use iterative rooting/postorder for a path of length 200000. Do not allow a child in state 2 when u has no guard, and do not accept state 1 unless at least one child actually has a guard.

```
One vertex.
Two vertices with very different costs.
A path, a star, and a branching tree.
A cheap child that can protect an expensive parent.
```

The oracle enumerates all guard subsets for small trees and checks coverage directly.

18.5 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    cin >> n;
    vector<long long> cost(n);
    for (long long& value : cost)
        cin >> value;

    vector<vector<int>> graph(n);
    for (int edge = 1; edge < n; ++edge) {
        int u, v;
        cin >> u >> v;
        --u;
        --v;
        graph[u].push_back(v);
        graph[v].push_back(u);
    }

    vector<int> parent(n, -2), order = {0};
    parent[0] = -1;
    for (int index = 0; index < n; ++index) {
        int u = order[index];
        for (int v : graph[u]) {
            if (v == parent[u])
                continue;
            parent[v] = u;
            order.push_back(v);
        }
    }
```

```

}

const long long INF = 4'000'000'000'000'000'000LL;
// State 0: u has a guard.
// State 1: u has no guard and is protected by a child.
// State 2: u is not protected yet and needs its parent to have a guard.
vector<array<long long, 3>> dp(n);

for (int index = n - 1; index >= 0; --index) {
    int u = order[index];
    long long selected = cost[u];
    long long covered_base = 0;
    long long force_guard_child = INF;
    long long awaiting_parent = 0;

    for (int v : graph[u]) {
        if (parent[v] != u)
            continue;

        selected += min({dp[v][0], dp[v][1], dp[v][2]});

        long long best_covered_child = min(dp[v][0], dp[v][1]);
        covered_base += best_covered_child;
        force_guard_child = min(force_guard_child,
                               dp[v][0] - best_covered_child);

        if (awaiting_parent == INF || dp[v][1] >= INF / 2)
            awaiting_parent = INF;
        else
            awaiting_parent += dp[v][1];
    }

    dp[u][0] = selected;
    dp[u][1] = (force_guard_child == INF
                ? INF : covered_base + force_guard_child);
    dp[u][2] = awaiting_parent;
}

// The root has no parent, so it cannot remain in state 2.
cout << min(dp[0][0], dp[0][1]) << '\n';
}

```

18.6 Full Python solution

```

import sys

def main() -> None:

```

```

data = list(map(int, sys.stdin.buffer.read().split()))
index = 0
n = data[index]
index += 1
cost = data[index:index + n]
index += n
graph = [[] for _ in range(n)]
for _ in range(n - 1):
    u, v = data[index] - 1, data[index + 1] - 1
    index += 2
    graph[u].append(v)
    graph[v].append(u)

parent = [-2] * n
parent[0] = -1
order = [0]
for u in order:
    for v in graph[u]:
        if v == parent[u]:
            continue
        parent[v] = u
        order.append(v)

infinity = 10**30
dp = [[0, 0, 0] for _ in range(n)]

for u in reversed(order):
    selected = cost[u]
    covered_base = 0
    force_guard_child = infinity
    awaiting_parent = 0

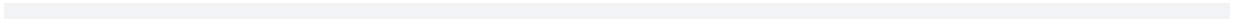
    for v in graph[u]:
        if parent[v] != u:
            continue
        selected += min(dp[v])
        best_covered_child = min(dp[v][0], dp[v][1])
        covered_base += best_covered_child
        force_guard_child = min(
            force_guard_child, dp[v][0] - best_covered_child)
        awaiting_parent += dp[v][1]

    dp[u][0] = selected
    dp[u][1] = covered_base + force_guard_child
    dp[u][2] = awaiting_parent

print(min(dp[0][0], dp[0][1]))

if __name__ == "__main__":
    main()

```



19 DP I closing checklist

After completing the twelve problems, check that you can answer these without looking at an implementation:

```
What does one state promise?  
What facts about the partial solution can affect choices made later?  
If I take any valid solution, is its last decision one of the choices tried by  
the recurrence?  
Does every choice tried by the recurrence preserve all the problem's rules?  
Which dependency determines the iteration order?  
Can the transition cost be reduced without changing the recurrence?  
Which old layer must remain untouched during an update?  
What is the smallest independent brute-force oracle?
```

The three middle questions check whether the state and transition are complete and safe:

- **What facts can affect later choices?** Imagine hiding the inside of the solved subproblem. The state must retain every fact that the unsolved part still needs to know. For Hamiltonian Route, these facts are the visited set and the current endpoint. For Weighted Tree Guards, the parent needs to know whether the subtree root has a guard, is already protected, or still needs protection from above.
- **Does the recurrence include every valid solution?** Take an arbitrary valid solution represented by the current state and inspect its final decision. Matrix Chain tries every possible final split, so whichever split that solution uses appears in the transition. This proves that the DP does not overlook a valid candidate.
- **Can a transition create an invalid solution?** Inspect each candidate produced by the recurrence and verify that it preserves every constraint. Hamiltonian Route adds only an unvisited vertex connected by an existing edge. Grouped Cargo reads from the previous group layer, so one transition cannot select two items from the same group. This proves that the DP does not optimize over forbidden candidates.

The second question is often called **completeness**: every valid solution is considered. The third is often called **soundness**: every considered solution is valid. A correct transition needs both.

If a recurrence is difficult to prove, return to the state sentence. Most DP proof failures begin because the state forgot a boundary condition or because its meaning changes between initialization, transition, and final answer.